RECEIVABLES RISK · METHOD NOTE

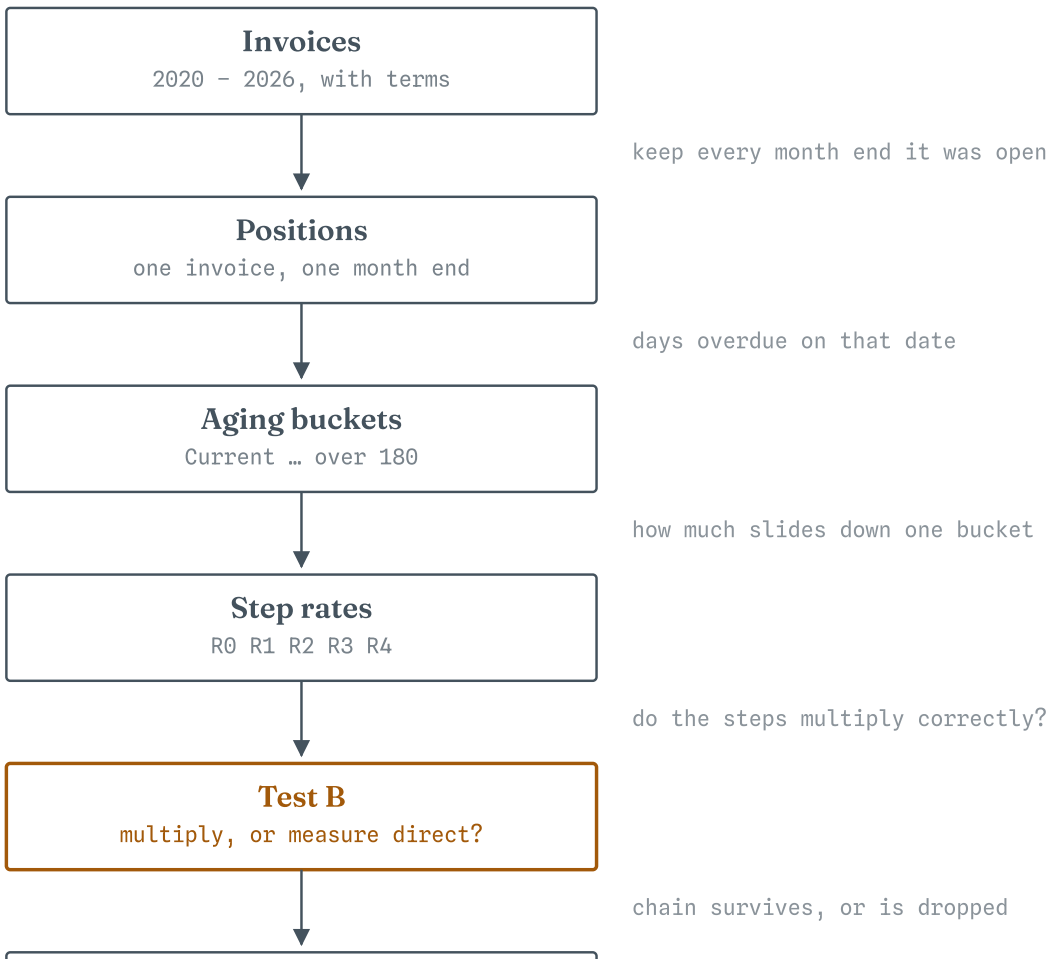
How the Rate Is Built

Six steps, from raw invoices to one rate per aging bucket. **The whole method rests on one idea that is easy to get wrong** — what counts as a single row. Step 01 is that idea; everything after it follows.

WHAT THE RATE MEASURES

Of the money sitting in this bucket today, what share will reach 180 days overdue?

Not credit loss. In eight years of payment records there is **no write-off transaction at all**, so loss is never recorded and cannot be measured. Reaching 180 days is something the data does record, so that is what the model claims.

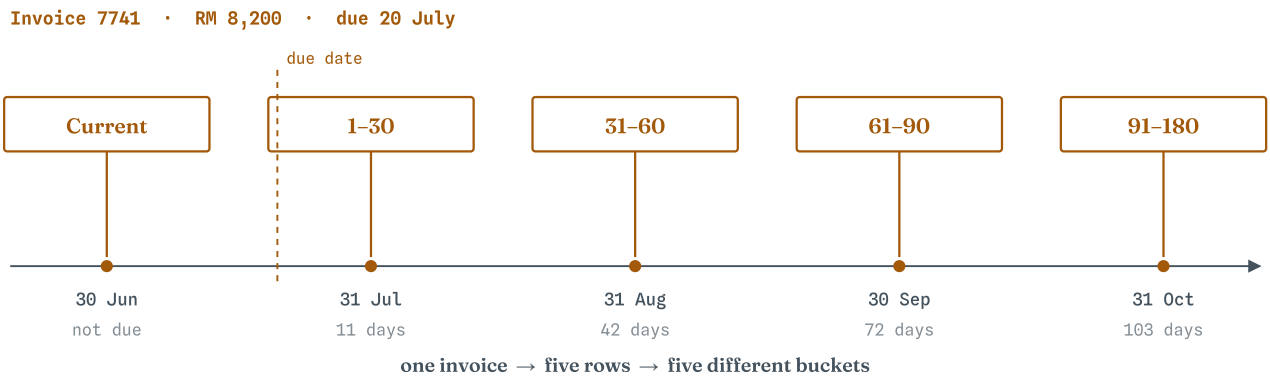


One rate per bucket
the number the report uses

The build, end to end. **Test B is the only decision point** — everything above it is measurement, everything below depends on its answer.

STEP 01 One row is one invoice at one month end

An invoice does not belong to a bucket. It belongs to a bucket *on a date*. The same invoice appears in a different bucket every month it stays unpaid, and **each of those appearances is its own row**.



The same RM 8,200 is measured five times, in five buckets, at five month ends. **It is never counted twice inside one bucket.**

This is where the volume comes from. Only about a thousand invoices are open at any month end — far too few to measure anything. Across roughly 75 month ends those thousand invoices produce **about 45,000 rows**, which is enough.

Closed invoices stay in

An invoice paid in 2022 still contributes rows — one for every month end it was open before that. You drop it only from the month ends *after* it closed. In the source data an open invoice carries `ClosedDateKey = -1`, not a null, so the test is `OpenInvoice = 1 OR MonthEnd < ClosedDate`.

STEP 02 Give every row a bucket

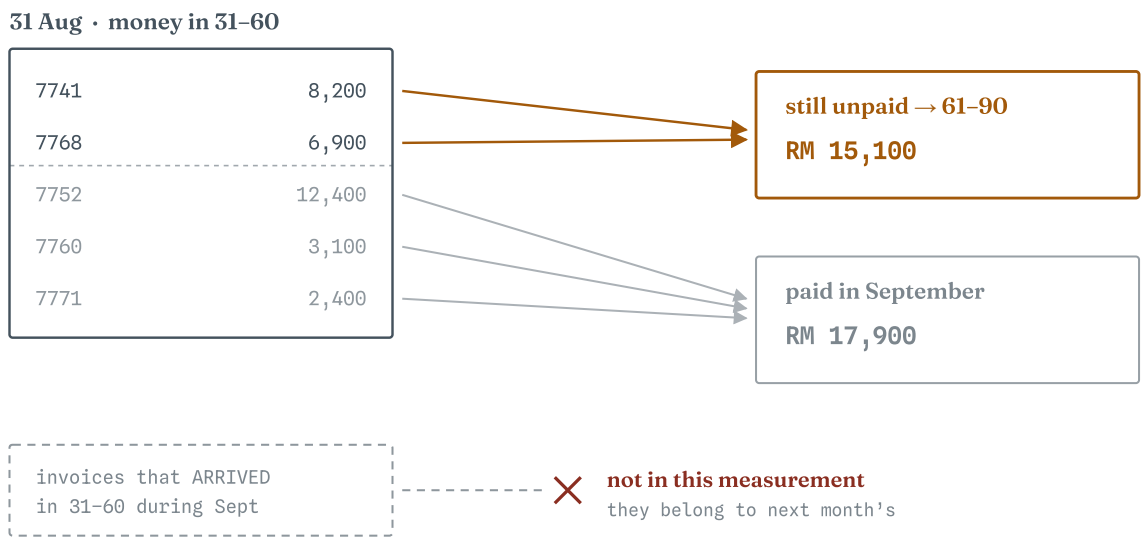
Days overdue at that month end decides the bucket. Nothing else.

BUCKET	DAYS OVERDUE	WIDTH	HOW IT IS MEASURED
Current	not yet due	varies	cohort — see step 05
1–30	1 to 30	30 days	monthly roll
31–60	31 to 60	30 days	monthly roll
61–90	61 to 90	30 days	monthly roll
91–180	91 to 180	90 days	cohort — see step 05
over 180	181 and up	–	already there, rate is 100%

The three middle buckets are each 30 days wide, so **one month of waiting equals exactly one step down**. That is what makes the monthly roll valid for them. The two shaded buckets break that rule, which is why they are handled separately.

STEP 03 Measure one step

Take the money in one bucket at one month end. Look again a month later.
How much of it is still unpaid, and so has slid down one bucket?



The denominator is **frozen on 31 August**. Money arriving in the bucket during September is excluded — it forms the next

observation instead.

```
roll rate  =  15,100  ÷  33,000  =  45.8%
```

That figure is high only because five rows is a toy example. A real month holds a few hundred rows. The mechanics are what matters: **every invoice in the bucket is in the denominator, including the ones that pay**. Money that behaves has to be counted, or the rate means nothing.

> **Only three buckets work this way**
>
> Monthly roll is valid where one month of waiting moves an invoice exactly one bucket. That is true for 1–30, 31–60 and 61–90, and false for Current and 91–180.

STEP 04 Pool the window — sum, then divide once

One month is not a rate; it is one reading. Collect every month in the window, add up both columns, and **divide a single time at the end**.

ILLUSTRATIVE AMOUNTS RATES ARE MEASURED

MONTH MEASURED	IN THE BUCKET	ROLLED DOWN	RATE
Jan	390,000	25,400	6.5%
Feb	410,000	22,600	5.5%
Mar	395,000	31,600	8.0%
Apr	405,000	20,300	5.0%
May	415,000	29,100	7.0%
Jun	390,000	23,400	6.0%
… six more months	…	…	…
Window total	**4,945,000**	**328,300**	**6.64%**

```
✓  328,300 ÷ 4,945,000  =  6.64%
✗  average the monthly percentages  —  weights a 390k month
```

```
    the same as a 415k month, which is wrong
```

Averaging the percentages treats a quiet month as equal to a heavy one. Pooling the money does not. The gap between the two is small in a steady year and **large in exactly the years you most need to get right**.

STEP 05 Two buckets need a different measurement

Current has no fixed width. How long an invoice stays there depends on its credit terms, and the terms differ by customer.

CREDIT TERMS	MONTH ENDS SPENT IN CURRENT	REAL CHANCES TO GO OVERDUE
30 days	~1	1
60 days	~2	1
120 days	~4	1

A 120-day invoice enters the denominator four times while only ever getting one real chance to fall overdue. **That drags the rate down for a reason that has nothing to do with the customer.**

Monthly roll answers

28.9%

Chance it goes overdue *this month*.

The chain needs

74.4%

Chance it *ever* goes overdue. Over four exposures at 28.9% a month, these are not close.

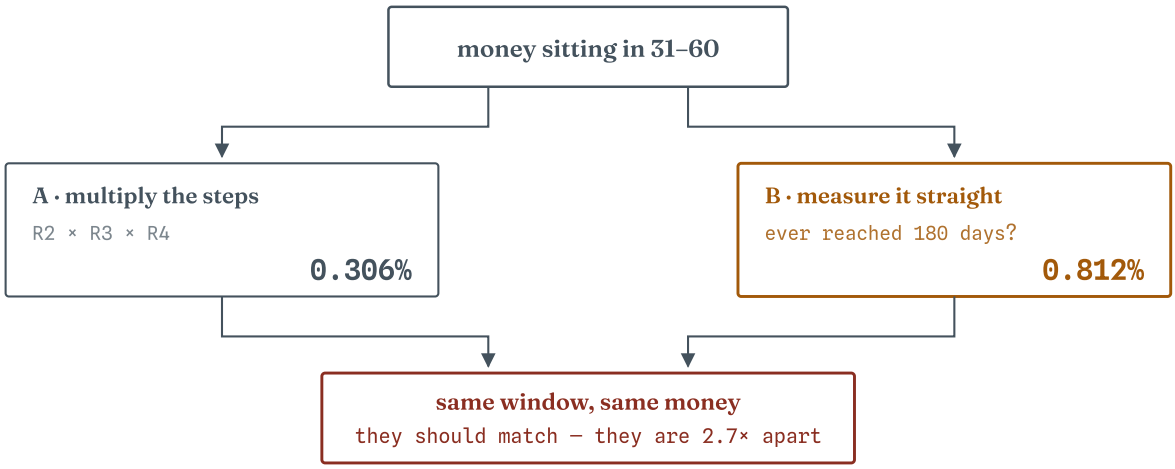
So Current is measured as a **cohort** instead — one observation per invoice, asking only whether it ever went past due. Terms length stops distorting it.

```
Current  =  value of invoices that EVER went past due
         ÷  value of all invoices
```

91–180 has the opposite problem: it is 90 days wide, so an invoice sits there for about three month ends. Measured as a single monthly roll it would be understated by roughly a third. It also uses a cohort.

STEP 06 Chain the steps, or measure straight?

There are two ways to turn step rates into the final number, and they should agree. **Checking whether they do is the cheapest useful test in the whole method.**



Both routes describe the same money over the same period, so no forecast is involved. **A gap means the multiplication is invalid**, not that the window is stale.

Multiplying assumes each step is independent of the one before it — that an invoice which already slid once is no more likely to slide again. In practice slow payers stay slow, so the assumption fails a little at every multiplication and compounds. The more steps, the wider the gap:

BUCKET	STEPS MULTIPLIED	CHAINED	DIRECT	GAP
91–180	1	18.18%	18.40%	+1%
61–90	2	4.61%	6.95%	+51%
31–60	3	0.306%	0.812%	+165%
1–30	4	0.0221%	0.0945%	+328%
Current	5	0.0094%	0.0611%	+550%

One multiplication is fine. Five is off by a factor of six. **If the test comes out like this, the fix is to delete the chain** — use the direct measurement and keep the step rates for diagnosis only. That is less code and one fewer

assumption.

> **Why this test is cheap**
>
> It needs no training and testing split. Both columns come from the same period, because the question is whether the arithmetic works, not whether it predicts. **Run it before building anything on top of the chain.**

RESULT

What the measurement produced

The 31–60 step, measured year by year across the usable history. Pooling the training years gives one rate; the yearly figures show why the window cannot be the whole history.

YEAR MEASURED	31–60 → 61–90	READING
2020	5.65%	
2021	6.60%	
2022	9.64%	
2023	1.97%	lowest on record
2024	9.50%	
2025	12.89%	
2026 to September	19.09%	nearly 10× 2023
Pooled 2020–2023 · 1,053 observations	**6.64%**	**the training rate**

A rate pooled over six years is **systematically too low for the book as it stands now**. That is not a flaw in the method; it is the reason the operating window has to be short and rolling rather than anchored to 2020.

> **Two steps cannot be measured at all**
>
> 61–90 and deeper carry roughly 150 observations across six years. No window and no split makes them readable. They are reported **without an error figure**, stating that the rate is carried from the pooled history.

HOLD ON TO Three rules

01	One row is **one invoice at one month end**. The same invoice appears in several buckets across its life, and never twice inside one bucket.
02	**Sum both columns, then divide once.** Never average the monthly percentages — that weights a quiet month like a heavy one.
03	A bucket may only use the monthly roll if **one month of waiting moves it exactly one bucket**. Current and 91–180 fail that test and use a cohort.

Rates and observation counts are measured from the invoice history, July 2020 to September 2026, with-terms invoices only. Amounts inside the worked examples are illustrative and chosen to show the mechanics. The chained and direct figures in step 06 are the comparison the method still has to confirm against the data.